

MANUAL DE PRACTICA

Modulo 2 — Sesion 1: Ingesta de Datos

Todo lo necesario para ejecutar las 4 practicas guiadas: archivos, scripts, API y pasos detallados en Microsoft Fabric.

1. Requisitos previos

- Workspace de Microsoft Fabric con capacidad activa (Trial gratuita o F-SKU/P-SKU). En app.fabric.microsoft.com → botón “Start trial” si no tienes una.
- Rol de Member o Contributor (o superior) en ese workspace.
- Navegador actualizado: Edge o Chrome.
- Los 3 archivos entregados junto a este manual: `ventas.csv`, `Setup_Warehouse_Origen.sql`, `Insertar_Filas_Nuevas.sql`.
- Conexión a internet estable (la Practica 3 llama una API pública en vivo).

Nota: No se necesita instalar nada en tu computadora. Notebooks, Pipelines y el editor SQL del Warehouse corren enteramente en el navegador.

2. Materiales entregados

Esta tabla resume cada archivo o recurso que usarás y en qué práctica entra:

Archivo / Recurso	Para que sirve	Se usa en
ventas.csv	Dataset de 60 filas de ventas (columnas: VentaID, Fecha, Cliente, Producto, Categoria, Cantidad, PrecioUnitario, Total, Region, Vendedor)	Practica 1
Setup_Warehouse_Origen.sql	Crea y llena la tabla <code>dbo.Ventas</code> que actua como el “SQL Server origen” del laboratorio	Practica 2
Insertar_Filas_Nuevas.sql	Agrega 8 filas nuevas en vivo, durante la clase, para demostrar carga incremental (watermark)	Practica 2
PokeAPI (publica)	API REST gratuita, sin autenticacion, con paginacion por cursor. Endpoint: https://pokeapi.co/api/v2/pokemon	Practica 3
Ingesta_de_Datos_Sesion1.pptx	Diapositivas teoricas de la sesion	Todas

3. Preparar el entorno (una sola vez, antes de empezar)

3.1 Crear el workspace

1. Entra a app.fabric.microsoft.com.
2. Workspaces → New workspace → nombralo Curso-Ingesta-Datos.
3. Asegúrate de asignarle una capacidad Fabric (Trial o la que tenga tu organización).

3.2 Crear los items de trabajo

Dentro del workspace crea estos 3 items (New item):

- Lakehouse → nombre: LH_Bronze_Ingesta (aquí llegan las 4 prácticas).
- Lakehouse → nombre: LH_Origen_Compartido (simula una fuente externa, solo para la Práctica 4 de Shortcuts).
- Warehouse → nombre: WH_Origen_Ventas (simula el SQL Server externo de la Práctica 2).

Nota: Usar un Warehouse de Fabric como “origen SQL” evita depender de un servidor externo o de una suscripción Azure aparte: expone un SQL connection string real y se conecta exactamente igual que un SQL Server o Azure SQL de producción. Si tu organización ya tiene una base SQL de prueba, puedes usarla en su lugar y saltar el Setup_Warehouse_Origen.sql.

4. Práctica 1 — Archivos planos → Lakehouse

Objetivo: cargar ventas.csv a Files, leerlo con PySpark y guardarlo como tabla Delta en Bronze.

Tiempo: ~25-30 min.

1. En LH_Bronze_Ingesta, panel Explorer → carpeta Files → New subfolder → nombre raw.
2. Dentro de Files/raw, botón Upload → selecciona ventas.csv.
3. En el workspace, New item → Notebook. Nombralo NB_Ingesta_Archivos.
4. En el panel izquierdo del notebook, Add lakehouse → selecciona LH_Bronze_Ingesta (para poder leer Files/ y escribir tablas).
5. Pega el siguiente código en la primera celda y ejecútala (Run):

```
df = (spark.read
      .option("header", True)
      .option("inferSchema", True)
      .csv("Files/raw/ventas.csv"))

df.printSchema()
df.show(5)
```

6. En una segunda celda, agrega las columnas de auditoria y escribe en Bronze:

```
from pyspark.sql.functions import current_timestamp, input_file_name

df_bronze = (df
             .withColumn("fecha_carga", current_timestamp())
             .withColumn("archivo_origen", input_file_name()))

df_bronze.write.format("delta").mode("append") \
             .saveAsTable("bronze.ventas")
```

7. Verifica en una celda nueva:

```
spark.sql("SELECT COUNT(*) AS filas FROM bronze.ventas").show()
```

Resultado esperado: 60 filas en bronze.ventas, visible también en el Explorer bajo Tables.

5. Práctica 2 — SQL Server → Lakehouse con Pipeline

Objetivo: usar un Data Pipeline (Copy Data) para traer datos desde una base SQL, y luego demostrar carga incremental. · Tiempo: ~30 min.

5.1 Cargar los datos “origen”

1. Abre WH_Origen_Ventas → pestaña New SQL query.
2. Pega el contenido completo de Setup_Warehouse_Origen.sql y ejecutalo (Run). Debe reportar 60 filas cargadas.

5.2 Obtener la cadena de conexión SQL

1. En WH_Origen_Ventas → Settings (icono de engranaje) → copia el “SQL connection string”.

5.3 Crear la conexión en Fabric

1. Workspace → Manage connections and gateways → New → tipo SQL Server.
2. Pega el servidor (SQL connection string) y la base (nombre del Warehouse). Autenticación: Organizational account (tu misma cuenta de Fabric).

5.4 Construir el pipeline

1. New item → Data pipeline → nombre PL_Ingesta_SQL_Ventas.
2. Agrega la actividad Copy data (assistant o manual).
3. Source: elige la conexión creada en 5.3, tabla dbo.Ventas (o usa Query y pega el filtro de abajo).
4. Destination: LH_Bronze_Ingesta → tabla nueva bronze_ventas_sql, modo Append.
5. Run. Verifica en Output que copió 60 filas.

5.5 Demostrar la carga incremental (en vivo, durante la clase)

1. Vuelve al SQL query editor de WH_Origen_Ventas y corre Insertar_Filas_Nuevas.sql (agrega 8 filas nuevas).
2. En el pipeline, edita el Source y usa esta consulta con filtro por fecha de modificación:

```
SELECT *
FROM dbo.Ventas
WHERE FechaModificacion > '@{formatDateTime(addhours(utcnow(), -2), 'yyyy-MM-dd
HH:mm:ss')}
```

3. Ejecuta el pipeline de nuevo: esta vez solo debería copiar las 8 filas nuevas, no las 60 anteriores.

Resultado esperado: bronze_ventas_sql termina con 68 filas, en dos corridas distintas (evidencia de full load + incremental).

6. Práctica 3 — Consumir una API REST desde Notebook

Objetivo: Llamar una API pública, paginar resultados, transformarlos y guardarlos en Bronze. · Tiempo: ~30 min.

Nota: API usada: PokeAPI (<https://pokeapi.co/api/v2/pokemon>). Es gratuita, no requiere API key ni registro, y no tiene límite de uso agresivo — ideal para un laboratorio en clase sin depender de credenciales externas. Pagina por cursor: cada respuesta trae un campo next con la URL de la siguiente página, hasta que next es null (patrón muy común en APIs reales, distinto al patrón page=1,2,3 usado en el ejemplo conceptual de las diapositivas).

1. New item → Notebook → nombre NB_Ingesta_API.
2. Add lakehouse → LH_Bronze_Ingesta.
3. Pega y ejecuta:

```
import requests

url = "https://pokeapi.co/api/v2/pokemon?limit=100"
resultados = []

while url:
    r = requests.get(url, timeout=30)
    r.raise_for_status()
    payload = r.json()
    resultados.extend(payload["results"])    # cada item: {"name": ..., "url": ...}
    url = payload["next"]                  # None cuando ya no hay mas paginas

print(f"Total registros: {len(resultados)}")
```

4. Convierte a DataFrame y guarda en Bronze:

```
from pyspark.sql.functions import current_timestamp

df_api = spark.createDataFrame(resultados)
df_api = df_api.withColumn("fecha_carga", current_timestamp())

df_api.write.format("delta").mode("overwrite") \
    .saveAsTable("bronze.api_pokemon")

spark.sql("SELECT COUNT(*) FROM bronze.api_pokemon").show()
```

Resultado esperado: mas de 1000 filas en bronze.api_pokemon.

Variante con autenticación (para mencionar, no obligatoria en clase)

Si una API real requiere API key, se agrega como header en la misma llamada:

```
headers = {"Authorization": "Bearer TU_TOKEN_AQUI"}
r = requests.get(url, headers=headers, timeout=30)
```

7. Práctica 4 — Crear un Shortcut en el Lakehouse

Objetivo: apuntar LH_Bronze_Ingesta a datos que viven en otro Lakehouse, sin copiarlos. · Tiempo: ~20 min.

1. En LH_Origen_Compartido, sube nuevamente ventas.csv a Files/raw (o reutiliza la tabla bronze.ventas si ya la creaste ahí también).
2. Ve a LH_Bronze_Ingesta → sección Tables → clic derecho → New shortcut.
3. Elige el tipo de origen: Microsoft OneLake.
4. Navega: workspace Curso-Ingesta-Datos → LH_Origen_Compartido → selecciona la tabla o carpeta a enlazar.
5. Confirma. El shortcut aparece en Tables con un icono de enlace (flecha), pero no ocupa storage adicional.
6. Pruébalo: `SELECT * FROM ese_shortcut LIMIT 10` — debe leer los datos en tiempo real desde LH_Origen_Compartido.

Nota: El mismo flujo aplica para ADLS Gen2, Amazon S3, Google Cloud Storage o Dataverse: solo cambia el tipo de origen elegido en el paso 3 y se pide una conexión con credenciales al recurso externo.

8. Checklist de cierre

Practica	Como verificar que salio bien	Resultado esperado
1. Archivos planos	Consultar <code>SELECT COUNT(*) FROM bronze.ventas</code> en el Lakehouse	60 filas
2. SQL + Pipeline	Revisar el historial de ejecuciones del pipeline: 2 corridas exitosas (full + incremental)	60 + 8 filas
3. API REST	Consultar <code>SELECT COUNT(*) FROM bronze.api_pokemon</code>	> 1000 filas
4. Shortcuts	La tabla/carpeta aparece en Tables con el icono de acceso directo (flecha)	Sin duplicar storage

Problemas comunes

- Error de permisos al crear la conexión SQL: confirma que tu cuenta tiene rol Viewer o superior en WH_Origen_Ventas.
- El notebook no encuentra Files/ o bronze.*: falta hacer Add lakehouse en el panel izquierdo del notebook.
- La Práctica 3 devuelve pocos resultados: revisa que url no se haya quedado en None desde la primera vuelta (imprime `payload["next"]` para depurar).
- El pipeline de la Práctica 2 no filtra filas nuevas: revisa que la columna FechaModificacion se esté actualizando (usa `SYSUTCDATETIME()` al insertar).